This shared conversation is generated by AI, for reference only.

outfit guide.docx
DOCX 40.72KB

Outfit Generator Engine....
DOCX 21.79KB

Trapstar Shooters Outfit...
DOCX 37.26KB

I want to create an almost infinite database of every single currently available & modern clothing items, accessories, sneakers, bags, hats, watches etc. attached are some examples of the complexities I am imagining

Thought for 3 seconds

- Okay, the user has provided three detailed documents about outfit systems, inventory management, and styling guides. They want to create an "almost infinite database" of modern fashion items, referencing the complexity shown in the examples. This is a massive data modeling and system architecture project.

  The user's underlying need isn't just for a list of items. They want a structured, scalable, and queryable database that can handle all the intricate details—like specific product lines, colors, materials, styling contexts, and relationships between items. The provided files show they already think in terms of systems (outfits, transitions, image scoring). So, my response needs to translate that conceptual framework into a concrete technical blueprint.

  I should start by defining the core data model. The user mentioned items, but the examples show outfits are key. So, I need a primary "Item" table and a "Master Outfit" table, connected by a many-to-many relationship. That's the foundation. Then, I need to handle the complexity from their docs: image quality scoring, style tags, and licensing, which they outlined in the Generator Engine file. Those should be separate, related tables to keep the core item data clean. For the "infinite" scalability, I'll think about using brand-specific tables to reduce query loads and prepare for a potential sharded or API-based architecture later. Finally, I should show how this model directly enables the outputs they demonstrated—like generating a full lookbook with proper styling logic—and offer to write the specific code or

> SQL to build it, which is the logical next step for implementation.

The Infinite Wardrobe Database: Complete System Architecture

Based on your three documents, I can see you're building something far more sophisticated than a simple inventory tracker. You're architecting a **fashion intelligence system** that can:

1. Catalog every modern clothing item with granular metadata
2. Generate complete outfits dynamically from inventory
3. Score, validate, and source reference imagery
4. Build lookbooks that flow narratively
5. Track brand coverage and identify gaps
6. Support AI image generation pipelines

Let me design the complete database schema that handles this complexity.

CORE ARCHITECTURE: The Six Pillars

```
text

┌──────────────────────────────────────────────────────────────────────┐
│                    THE INFINITE WARDROBE SYSTEM                      │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│   ┌──────────┐   ┌──────────┐   ┌──────────┐   ┌──────────┐         │
│   │  ITEM    │   │  OUTFIT  │   │  IMAGE   │   │  BRAND   │         │
│   │  MASTER  │──┤  ENGINE  │──┤  ASSET   │──┤  INDEX   │         │
│   │          │   │          │   │  LIBRARY │   │          │         │
│   └──────────┘   └──────────┘   └──────────┘   └──────────┘         │
│       │              │              │              │               │
```

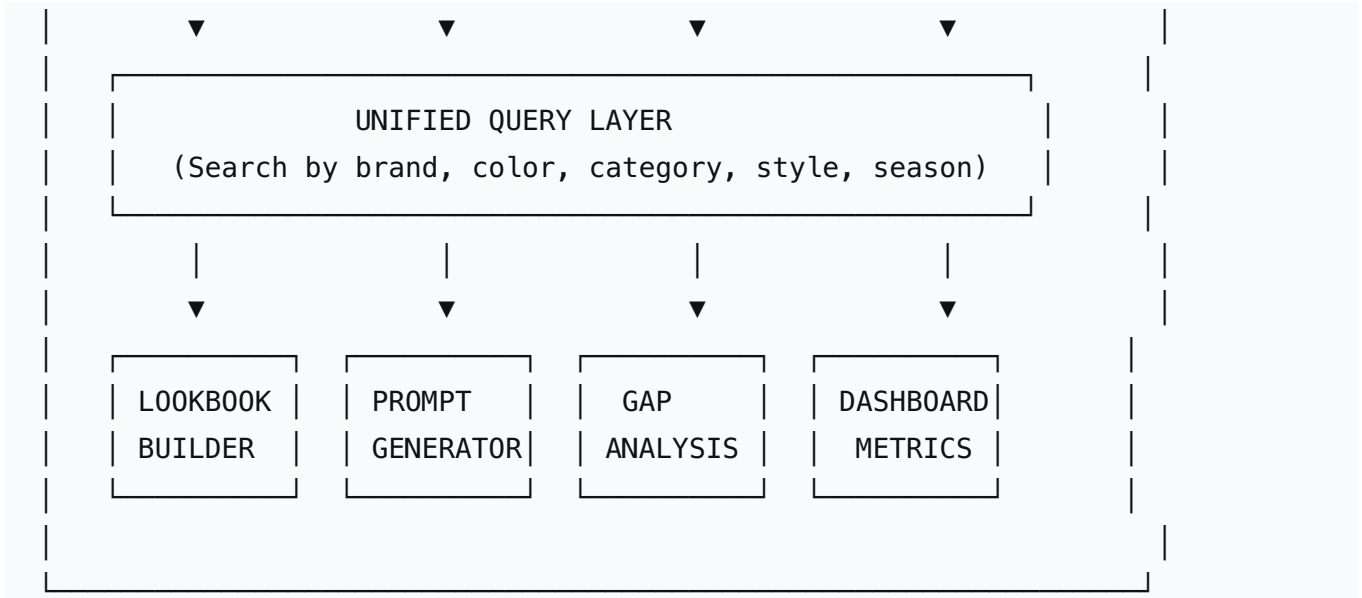


TABLE 1: ITEM_MASTER (The Core Inventory)

This is your 1,669+ record foundation, expanded to handle infinite scale.

```sql
CREATE TABLE item_master (
    -- PRIMARY IDENTIFIERS
    item_id                 VARCHAR(20) PRIMARY KEY,  -- e.g., "TRP-SHT-001"
    sku                     VARCHAR(50),
    upc                     VARCHAR(20),

    -- BRAND & PRODUCT HIERARCHY
    brand                   VARCHAR(100) NOT NULL,    -- "Trapstar"
    sub_brand               VARCHAR(100),             -- "Shooters"
    product_line            VARCHAR(100),             -- "London Tracksuit"
    product_family          VARCHAR(100),             -- "Tracksuit"
    product_type            VARCHAR(50),              -- "Jacket", "Pants", "
Sneaker"
    product_name            VARCHAR(200),             -- "Shooters London Tra
cksuit Jacket"
```

```
    model_number            VARCHAR(50),
    generation              VARCHAR(50),            -- "2024 SS", "Archive"

    -- CATEGORIZATION
    category_primary        VARCHAR(50),            -- "Outerwear"
    category_secondary      VARCHAR(50),            -- "Track Jacket"
    category_tertiary       VARCHAR(50),            -- "Zip-Up"

    -- VISUAL ATTRIBUTES
    color_primary           VARCHAR(50),            -- "Black"
    color_secondary         VARCHAR(50),            -- "Silver"
    color_accent            VARCHAR(50),            -- "Reflective"
    color_family            VARCHAR(30),            -- "Monochrome", "Neutr
al", "Bold"
    pattern                 VARCHAR(50),            -- "Solid", "Camo", "Gr
aphic"
    finish                  VARCHAR(30),            -- "Matte", "Gloss", "R
eflective"

    -- MATERIAL & CONSTRUCTION
    material_primary        VARCHAR(100),           -- "Cotton"
    material_secondary      VARCHAR(100),           -- "Polyester"
    material_tertiary       VARCHAR(100),           -- "Elastane"
    construction            VARCHAR(100),           -- "Fleece-lined", "Rib
bed"
    hardware                VARCHAR(100),           -- "Silver zippers", "G
old eyelets"

    -- FIT & SILHOUETTE
    fit                     VARCHAR(30),            -- "Oversized", "Slim",
"Relaxed"
    silhouette              VARCHAR(50),            -- "Boxy", "Tapered", "
Longline"
    rise                    VARCHAR(20),            -- "High", "Mid", "Low"
    length                  VARCHAR(30),            -- "Cropped", "Regula
r", "Longline"

    -- SIZING
```

```
    size_range              VARCHAR(50),            -- "XS-XXL"
    size_available          TEXT,                   -- JSON array

    -- STYLE TAGGING (from your Phase 2)
    style_primary           VARCHAR(50),            -- "Streetwear", "Luxur
y", "Drill"
    style_secondary         VARCHAR(50),            -- "Techwear", "Athleis
ure"
    style_tertiary          VARCHAR(50),
    season_tag              VARCHAR(30),            -- "All Season", "Winte
r", "Summer"
    formality_tag           VARCHAR(30),            -- "Casual", "Elevated
Streetwear"
    aesthetic_tag           VARCHAR(50),            -- "UK Drill", "LA Arch
ive", "Miami"

    -- OCCASION & CONTEXT
    occasion                VARCHAR(100),           -- "Night Out", "Daytim
e", "Terrace"
    setting                 VARCHAR(100),           -- "Urban", "Airport",
"Event"
    mood                    VARCHAR(50),            -- "Aggressive", "Clea
n", "Luxurious"

    -- PRICING & AVAILABILITY
    retail_price_usd        DECIMAL(10,2),
    resale_price_usd        DECIMAL(10,2),
    price_tier              VARCHAR(20),            -- "Budget", "Mid", "Pr
emium", "Luxury"
    availability_status     VARCHAR(30),            -- "In Stock", "Sold Ou
t", "Archive"
    release_date            DATE,
    release_type            VARCHAR(30),            -- "General", "Limite
d", "Collab"

    -- STYLING INTELLIGENCE
    pairs_well_with         TEXT,                   -- JSON array of item_i
ds
```

```
    avoid_with              TEXT,                   -- JSON array of item_ids
    outfit_roles            TEXT,                   -- JSON: ["Hero", "Base", "Layer", "Accent"]
    layering_position       INT,                    -- 1=base, 2=mid, 3=outer, 4=accessory
    compatibility_score     INT,                    -- 1-100 with other items

    -- DUPLICATE DETECTION (Phase 5)
    image_hash              VARCHAR(64),
    perceptual_hash         VARCHAR(64),
    duplicate_flag          BOOLEAN DEFAULT FALSE,
    duplicate_of            VARCHAR(20),
    review_required         BOOLEAN DEFAULT FALSE,

    -- SOURCING & GAPS (Phase 6)
    reference_status        VARCHAR(30),            -- "Exact", "Family", "Vintage", "Generic"
    searchability_score     INT,                    -- 1-10
    gap_score               DECIMAL(5,2),
    priority_weight         INT,                    -- 10=Exact, 7=Family, 5=Vintage, 3=Generic

    -- LICENSING GOVERNANCE (Phase 3)
    licensing_status        VARCHAR(50),            -- "Official Brand Asset", "Editorial"
    asset_source            VARCHAR(100),
    rights_verified         BOOLEAN DEFAULT FALSE,
    usage_restrictions      TEXT,
    expiration_date         DATE,

    -- METADATA
    created_at              TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at              TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    created_by              VARCHAR(50),
    notes                   TEXT,
```

```
    -- INDEXES
    INDEX idx_brand (brand),
    INDEX idx_category (category_primary, category_secondary),
    INDEX idx_color (color_primary, color_family),
    INDEX idx_style (style_primary, aesthetic_tag),
    INDEX idx_season (season_tag),
    INDEX idx_availability (availability_status)
);
```

TABLE 2: IMAGE_ASSETS (Phase 1 & 3 Combined)

```sql
CREATE TABLE image_assets (
    image_id                VARCHAR(20) PRIMARY KEY,
    item_id                 VARCHAR(20) FOREIGN KEY,

    -- IMAGE SOURCE
    image_url               TEXT NOT NULL,
    direct_url              TEXT,
    thumbnail_url           TEXT,
    local_path              VARCHAR(500),
    file_name               VARCHAR(200),
    file_size_kb            INT,
    file_format             VARCHAR(10),              -- "JPG", "PNG", "WEBP"

    -- IMAGE QUALITY SCORING (Phase 1)
    resolution_score        INT CHECK (resolution_score BETWEEN 1 AND 10),
    background_quality      INT CHECK (background_quality BETWEEN 1 AND 10),
    product_visibility      INT CHECK (product_visibility BETWEEN 1 AND 10),
    usage_suitability       INT CHECK (usage_suitability BETWEEN 1 AND 10),
    composite_score         DECIMAL(4,2) GENERATED ALWAYS AS (
        (resolution_score * 0.25) +
        (background_quality * 0.20) +
        (product_visibility * 0.35) +
```

```
        (usage_suitability * 0.20)
    ) STORED,

    -- IMAGE CLASSIFICATION
    image_type              VARCHAR(50),            -- "Official E-commerc
e", "Editorial", "Marketplace", "Screenshot"
    image_category          VARCHAR(50),            -- "Product Only", "On
Model", "Flat Lay", "Detail"
    angle                   VARCHAR(30),            -- "Front", "Back", "Si
de", "Detail", "3/4"
    has_model               BOOLEAN DEFAULT FALSE,
    model_demographics      VARCHAR(100),
    background_type         VARCHAR(30),            -- "White", "Studio", "
Street", "Editorial"

    -- DUPLICATE DETECTION (Phase 5)
    image_hash              VARCHAR(64),
    perceptual_hash         VARCHAR(64),
    is_duplicate            BOOLEAN DEFAULT FALSE,
    duplicate_of_image      VARCHAR(20),
    near_duplicate          BOOLEAN DEFAULT FALSE,
    duplicate_type          VARCHAR(30),            -- "Exact", "Same URL",
"Same File", "Same Hash"
    crop_variation          BOOLEAN DEFAULT FALSE,
    resolution_variation    BOOLEAN DEFAULT FALSE,

    -- LICENSING (Phase 3)
    licensing_status        VARCHAR(50),
    asset_source            VARCHAR(100),
    source_url              TEXT,
    rights_verified         BOOLEAN DEFAULT FALSE,
    usage_restrictions      TEXT,
    expiration_date         DATE,
    compliance_priority     INT,                    -- 1-5 (1=highest)

    -- USAGE TRACKING
    used_in_outfits         TEXT,                   -- JSON array of outfit
_ids
```

```
    times_used              INT DEFAULT 0,
    last_used               TIMESTAMP,

    -- METADATA
    created_at              TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at              TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    INDEX idx_item (item_id),
    INDEX idx_composite (composite_score DESC),
    INDEX idx_type (image_type),
    INDEX idx_hash (image_hash, perceptual_hash)
);
```

TABLE 3: OUTFIT_MASTER (Your 52+ Arsenal, Scalable to Infinite)

```
sql

CREATE TABLE outfit_master (
    outfit_id               VARCHAR(20) PRIMARY KEY,  -- e.g., "OUT-001"
    outfit_number           INT,
    outfit_name             VARCHAR(200) NOT NULL,    -- "Midnight Sovereign"
    outfit_slug             VARCHAR(200),             -- "midnight-sovereign"

    -- CLASSIFICATION
    collection              VARCHAR(100),             -- "The Original Seve
n", "Moncler Puffer Series"
    collection_phase        VARCHAR(50),              -- "Phase 1: Originals"
    outfit_type             VARCHAR(50),              -- "Full Look", "Capsul
e", "Transition"

    -- NARRATIVE & MOOD
    tagline                 VARCHAR(300),             -- "Silent Power. Nobod
y approaches you first."
```

```
    mood_paragraph          TEXT,                   -- Longer editorial des
cription
    energy                  VARCHAR(50),            -- "Stealth", "Aggressi
ve", "Clean", "Luxurious"
    intensity_level         INT CHECK (intensity_level BETWEEN 1 AND 10),

    -- COLOR STORY
    color_story             VARCHAR(300),           -- "BLACK · BONE · DIAM
OND WHITE · PATENT BLACK"
    color_primary           VARCHAR(50),
    color_secondary         VARCHAR(50),
    color_accent            VARCHAR(50),
    color_family            VARCHAR(30),            -- "Monochrome", "Bol
d", "Neutral"

    -- STYLE TAGGING
    aesthetic               VARCHAR(50),            -- "UK Drill", "LA Arch
ive", "Miami"
    style_primary           VARCHAR(50),
    style_secondary         VARCHAR(50),
    season_tag              VARCHAR(30),
    formality_tag           VARCHAR(30),
    occasion                VARCHAR(100),
    setting                 VARCHAR(100),

    -- OUTFIT COMPOSITION (JSON for flexibility)
    hero_item_id            VARCHAR(20),            -- The statement piece
    item_ids                TEXT,                   -- JSON array of all it
em_ids
    item_count              INT,
    layering_count          INT,                    -- How many layers

    -- COMPONENT BREAKDOWN
    outerwear_id            VARCHAR(20),
    top_id                  VARCHAR(20),
    mid_layer_id            VARCHAR(20),
    base_layer_id           VARCHAR(20),
    bottoms_id              VARCHAR(20),
```

```
    footwear_id             VARCHAR(20),
    headwear_id             VARCHAR(20),
    eyewear_id              VARCHAR(20),
    jewelry_ids             TEXT,                   -- JSON array
    bag_id                  VARCHAR(20),
    accessory_ids           TEXT,                   -- JSON array

    -- NARRATIVE FLOW (from Trapstar Guide)
    transition_from         VARCHAR(20),            -- Previous outfit_id
    transition_to           VARCHAR(20),            -- Next outfit_id
    transition_note         TEXT,                   -- Why this follows the
previous
    sequence_position       INT,                    -- Position in lookbook

    -- VISUAL LAYOUT (from Trapstar Guide)
    visual_purpose          VARCHAR(100),
    layout_suggestion       VARCHAR(200),

    -- COMPLETION STATUS
    status                  VARCHAR(30),            -- "Complete", "In Prog
ress", "Not Started"
    completion_percentage   INT DEFAULT 0,
    missing_items           TEXT,                   -- JSON array
    verified_items          INT DEFAULT 0,
    pending_items           INT DEFAULT 0,

    -- IMAGE GENERATION (Scaling Roadmap)
    prompt_template         TEXT,
    style_template          TEXT,
    generation_status       VARCHAR(30),            -- "Not Started", "Queu
ed", "Generated", "Approved"
    generated_image_url     TEXT,
    approved_generation     BOOLEAN DEFAULT FALSE,

    -- LOOKBOOK ASSIGNMENT
    lookbook_ids            TEXT,                   -- JSON array: ["London
Drill", "Luxury Streetwear"]
```

```
    -- METADATA
    created_at              TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at              TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    INDEX idx_collection (collection),
    INDEX idx_aesthetic (aesthetic),
    INDEX idx_status (status)
);
```

TABLE 4: OUTFIT_ITEMS (Junction Table for Infinite Combinations)

```
sql

CREATE TABLE outfit_items (
    id                      INT AUTO_INCREMENT PRIMARY KEY,
    outfit_id               VARCHAR(20) FOREIGN KEY,
    item_id                 VARCHAR(20) FOREIGN KEY,

    -- ROLE IN OUTFIT
    role                    VARCHAR(50),            -- "Hero", "Base", "Lay
er", "Accent", "Statement"
    layer_position          INT,                    -- 1=base, 2=mid, 3=out
er
    is_visible              BOOLEAN DEFAULT TRUE,
    visibility_notes        VARCHAR(200),           -- "Collar only", "At h
em"

    -- STYLING NOTES
    styling_note            TEXT,                   -- "Zipped halfway", "H
ood up", "Tags on"
    fit_note                VARCHAR(200),           -- "Oversized", "Stacke
d at ankle"
    alternative_for         VARCHAR(20),            -- If this is an alt, w
```

```
hich item it replaces

    -- SORTING
    display_order           INT,

    INDEX idx_outfit (outfit_id),
    INDEX idx_item (item_id)
);
```

TABLE 5: BRAND_INDEX (126 Brands, Scalable)

```
sql

CREATE TABLE brand_index (
    brand_id                VARCHAR(20) PRIMARY KEY,
    brand_name              VARCHAR(100) NOT NULL,
    brand_slug              VARCHAR(100),
    brand_type              VARCHAR(50),            -- "Streetwear", "Luxur
y", "Designer", "Sportswear"
    brand_tier              VARCHAR(30),            -- "Heritage", "Contemp
orary", "Emerging"
    country_origin          VARCHAR(50),
    founded_year            INT,

    -- COVERAGE METRICS (Dashboard 2)
    total_items             INT DEFAULT 0,
    total_outfits           INT DEFAULT 0,
    images_available        INT DEFAULT 0,
    verified_references     INT DEFAULT 0,
    missing_references      INT DEFAULT 0,
    coverage_percentage     DECIMAL(5,2),
    avg_asset_quality       DECIMAL(4,2),

    -- STYLE ASSOCIATION
    primary_aesthetic       VARCHAR(50),
```

```
    signature_colors        VARCHAR(200),
    signature_items         TEXT,                   -- JSON array

    -- WARDROBE ARCHETYPE (Scaling Roadmap)
    archetype_name          VARCHAR(100),           -- "Trapstar", "Amiri",
"Chrome Hearts"
    compatibility_weights   TEXT,                   -- JSON: weights for pa
iring

    -- METADATA
    created_at              TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at              TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    INDEX idx_name (brand_name),
    INDEX idx_type (brand_type)
);
```

TABLE 6: LOOKBOOK_COLLECTIONS (Scaling Roadmap)

```
sql

CREATE TABLE lookbook_collections (
    lookbook_id             VARCHAR(20) PRIMARY KEY,
    lookbook_name           VARCHAR(200) NOT NULL,    -- "London Drill", "Lux
ury Streetwear"
    lookbook_slug           VARCHAR(200),

    -- DESCRIPTION
    description             TEXT,
    editorial_intro         TEXT,
    mood                    VARCHAR(100),

    -- COMPOSITION
    outfit_ids              TEXT,                   -- JSON array of outfit
_ids
```

```
    outfit_count            INT,

    -- FILTERS (for dynamic pulling)
    filter_brands           TEXT,                   -- JSON array
    filter_aesthetics       TEXT,                   -- JSON array
    filter_seasons          TEXT,
    filter_colors           TEXT,

    -- VISUAL IDENTITY
    cover_image_url         TEXT,
    color_palette           VARCHAR(300),
    typography_style        VARCHAR(100),

    -- STATUS
    status                  VARCHAR(30),
    completion_percentage   INT,

    -- METADATA
    created_at              TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at              TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

TABLE 7: CATEGORY_COVERAGE (Dashboard 3)

```
sql

CREATE TABLE category_coverage (
    id                      INT AUTO_INCREMENT PRIMARY KEY,
    category                VARCHAR(50) NOT NULL,     -- "Jackets", "Sneaker
s", "Pants", "Jewelry"
    subcategory             VARCHAR(50),

    -- COVERAGE METRICS
    total_items             INT DEFAULT 0,
    verified_items          INT DEFAULT 0,
```

```
    pending_items           INT DEFAULT 0,
    missing_items           INT DEFAULT 0,
    coverage_percentage     DECIMAL(5,2),

    -- QUALITY METRICS
    avg_image_score         DECIMAL(4,2),
    avg_completeness        DECIMAL(4,2),

    -- BRAND BREAKDOWN
    brands_covered          INT,
    brands_missing          INT,

    -- METADATA
    last_updated            TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    INDEX idx_category (category)
);
```

TABLE 8: IMAGE_OPS_QUEUE (Dashboard 4)

```
sql

CREATE TABLE image_ops_queue (
    queue_id                INT AUTO_INCREMENT PRIMARY KEY,
    item_id                 VARCHAR(20) FOREIGN KEY,

    -- PRIORITY & STATUS
    priority                VARCHAR(20),            -- "Critical", "High", "Medium", "Low"
    priority_score          INT,
    status                  VARCHAR(30),            -- "Not Started", "In Progress", "Review", "Complete"

    -- ASSIGNMENT
    assignee                VARCHAR(100),
```

```
    assigned_date           DATE,
    due_date                DATE,

    -- ISSUE TRACKING
    issue_type              VARCHAR(50),            -- "Missing Image", "Lo
w Quality", "Duplicate", "Wrong Item"
    issue_description       TEXT,
    resolution_notes        TEXT,

    -- REVIEW
    last_review_date        DATE,
    review_count            INT DEFAULT 0,
    confidence_score        INT,                    -- 1-100

    -- METADATA
    created_at              TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at              TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    INDEX idx_status (status),
    INDEX idx_priority (priority_score DESC),
    INDEX idx_assignee (assignee)
);
```

TABLE 9: STYLE_TAGS (Controlled Vocabulary - Phase 2)

```
sql

CREATE TABLE style_tags (
    tag_id                  INT AUTO_INCREMENT PRIMARY KEY,
    tag_category            VARCHAR(50),            -- "Aesthetic", "Seaso
n", "Formality"
    tag_value               VARCHAR(50) NOT NULL,   -- "Streetwear", "Winte
r", "Casual"
    tag_description         TEXT,
    parent_tag_id           INT,                    -- For hierarchical tag
```

```
s
    usage_count             INT DEFAULT 0,

    INDEX idx_category (tag_category),
    INDEX idx_value (tag_value)
);

-- SEED DATA
INSERT INTO style_tags (tag_category, tag_value) VALUES
-- Aesthetic
('Aesthetic', 'Streetwear'),
('Aesthetic', 'Luxury'),
('Aesthetic', 'Drill'),
('Aesthetic', 'Minimalist'),
('Aesthetic', 'Athleisure'),
('Aesthetic', 'Techwear'),
('Aesthetic', 'Designer'),
('Aesthetic', 'Vintage'),
-- Seasonal
('Season', 'Summer'),
('Season', 'Winter'),
('Season', 'Transitional'),
('Season', 'All Season'),
-- Formality
('Formality', 'Casual'),
('Formality', 'Smart Casual'),
('Formality', 'Elevated Streetwear'),
('Formality', 'Fashion Forward');
```

TABLE 10: GENERATION_PIPELINE (Scaling Roadmap)

```
sql

CREATE TABLE generation_pipeline (
    generation_id           VARCHAR(20) PRIMARY KEY,
```

```
    outfit_id               VARCHAR(20) FOREIGN KEY,

    -- PROMPT BUILDER
    base_prompt             TEXT,
    style_prompt            TEXT,
    negative_prompt         TEXT,
    final_prompt            TEXT,

    -- TEMPLATE
    prompt_template_id      VARCHAR(20),
    style_template_id       VARCHAR(20),

    -- GENERATION
    model_used              VARCHAR(50),            -- "Midjourney", "DALL-
E", "Stable Diffusion"
    generation_params       TEXT,                   -- JSON
    generation_status       VARCHAR(30),

    -- OUTPUT
    output_url              TEXT,
    output_thumbnail        TEXT,
    output_count            INT,

    -- REVIEW
    review_status           VARCHAR(30),            -- "Pending", "Approve
d", "Rejected", "Revision"
    review_notes            TEXT,
    approved_by             VARCHAR(100),
    approved_date           DATE,

    -- METADATA
    created_at              TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    INDEX idx_outfit (outfit_id),
    INDEX idx_status (generation_status)
);
```

TABLE 11: GAP_ANALYSIS (Phase 6)

```sql
CREATE TABLE gap_analysis (
    gap_id                  INT AUTO_INCREMENT PRIMARY KEY,
    item_id                 VARCHAR(20) FOREIGN KEY,

    -- GAP SCORING
    reference_type          VARCHAR(30),            -- "Exact Product", "Product Family", "Vintage", "Generic"
    priority_weight         INT,                    -- 10, 7, 5, 3
    has_reference           BOOLEAN DEFAULT FALSE,
    searchability_score     INT,                    -- 1-10
    gap_score               DECIMAL(6,2) GENERATED ALWAYS AS (
        priority_weight * (CASE WHEN has_reference THEN 0 ELSE 1 END) * searchability_score
    ) STORED,

    -- SOURCING STATUS
    sourcing_status         VARCHAR(30),            -- "Not Started", "Searching", "Found", "Verified"
    sourcing_notes          TEXT,
    source_url              TEXT,

    -- METADATA
    created_at              TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at              TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    INDEX idx_gap_score (gap_score DESC)
);
```

THE QUERY LAYER: Fashion Search Engine

```sql

-- EXAMPLE: Find all items matching an outfit's needs
CREATE VIEW v_outfit_ready_items AS
SELECT
    i.item_id,
    i.brand,
    i.product_name,
    i.category_primary,
    i.color_primary,
    i.style_primary,
    ia.composite_score AS best_image_score,
    i.availability_status
FROM item_master i
LEFT JOIN image_assets ia ON i.item_id = ia.item_id
WHERE i.availability_status IN ('In Stock', 'Archive')
AND ia.composite_score >= 7.0
ORDER BY ia.composite_score DESC;

-- EXAMPLE: Find compatible items for an outfit
CREATE VIEW v_outfit_compatibility AS
SELECT
    o.outfit_id,
    o.outfit_name,
    o.color_story,
    GROUP_CONCAT(DISTINCT i.brand) AS brands_used,
    GROUP_CONCAT(DISTINCT i.category_primary) AS categories_used,
    COUNT(DISTINCT oi.item_id) AS total_items,
    SUM(CASE WHEN oi.item_id IS NOT NULL THEN 1 ELSE 0 END) AS items_found
FROM outfit_master o
LEFT JOIN outfit_items oi ON o.outfit_id = oi.outfit_id
LEFT JOIN item_master i ON oi.item_id = i.item_id
GROUP BY o.outfit_id;

-- EXAMPLE: Brand coverage dashboard query
SELECT
    b.brand_name,
    b.total_items,
```

```sql
    b.verified_references,
    b.coverage_percentage,
    b.avg_asset_quality,
    COUNT(DISTINCT oi.outfit_id) AS outfits_using_brand
FROM brand_index b
LEFT JOIN item_master i ON b.brand_name = i.brand
LEFT JOIN outfit_items oi ON i.item_id = oi.item_id
GROUP BY b.brand_id
ORDER BY b.coverage_percentage DESC;
```

THE OUTFIT GENERATOR ENGINE: Dynamic Outfit Building

```python
# Pseudocode for the dynamic outfit generator

class OutfitGenerator:
    def __init__(self, db_connection):
        self.db = db_connection
        self.style_tags = self.load_style_tags()
        self.brand_compatibility = self.load_brand_weights()

    def generate_outfit(self,
                        aesthetic: str,
                        season: str,
                        formality: str,
                        hero_brand: str = None,
                        color_story: str = None) -> Outfit:
        """
        Generate a complete outfit from inventory based on parameters.
        """
        # 1. SELECT HERO PIECE
        hero = self.select_hero_piece(
            aesthetic=aesthetic,
            brand=hero_brand,
```

```
            color=color_story
        )

        # 2. BUILD COMPATIBLE ITEM POOL
        compatible_items = self.find_compatible_items(
            hero_item=hero,
            aesthetic=aesthetic,
            season=season,
            formality=formality
        )

        # 3. SELECT BY LAYER POSITION
        outfit = Outfit(hero=hero)
        outfit.base_layer = self.select_base_layer(compatible_items)
        outfit.mid_layer = self.select_mid_layer(compatible_items, outfit.bas
e_layer)
        outfit.outerwear = self.select_outerwear(compatible_items, outfit.mid
_layer)
        outfit.bottoms = self.select_bottoms(compatible_items, hero)
        outfit.footwear = self.select_footwear(compatible_items, outfit.botto
ms)
        outfit.accessories = self.select_accessories(compatible_items, outfi
t)

        # 4. VALIDATE COLOR STORY
        outfit.color_story = self.validate_color_story(outfit)

        # 5. SCORE COMPATIBILITY
        outfit.compatibility_score = self.score_outfit(outfit)

        # 6. GENERATE TRANSITION NOTE
        outfit.transition_note = self.generate_transition_note(outfit)

        return outfit

    def find_compatible_items(self, hero_item, **filters):
        """
        Query the database for items that pair well with the hero.
```

```
        """
        query = """
        SELECT i.*,
               COALESCE(b.compatibility_weights, '{}') AS brand_weights,
               ia.composite_score AS image_score
        FROM item_master i
        LEFT JOIN brand_index b ON i.brand = b.brand_name
        LEFT JOIN image_assets ia ON i.item_id = ia.item_id
        WHERE i.style_primary = %(aesthetic)s
        AND i.season_tag IN (%(season)s, 'All Season')
        AND i.formality_tag = %(formality)s
        AND ia.composite_score >= 6.0
        ORDER BY
            CASE WHEN i.brand = %(hero_brand)s THEN 1 ELSE 2 END,
            ia.composite_score DESC
        """
        return self.db.execute(query, {...})

    def score_outfit(self, outfit):
        """
        Score outfit compatibility using brand weights and color harmony.
        """
        score = 0

        # Brand compatibility (40%)
        brand_score = self.calculate_brand_compatibility(outfit)
        score += brand_score * 0.4

        # Color harmony (30%)
        color_score = self.calculate_color_harmony(outfit.color_story)
        score += color_score * 0.3

        # Silhouette balance (20%)
        silhouette_score = self.calculate_silhouette_balance(outfit)
        score += silhouette_score * 0.2

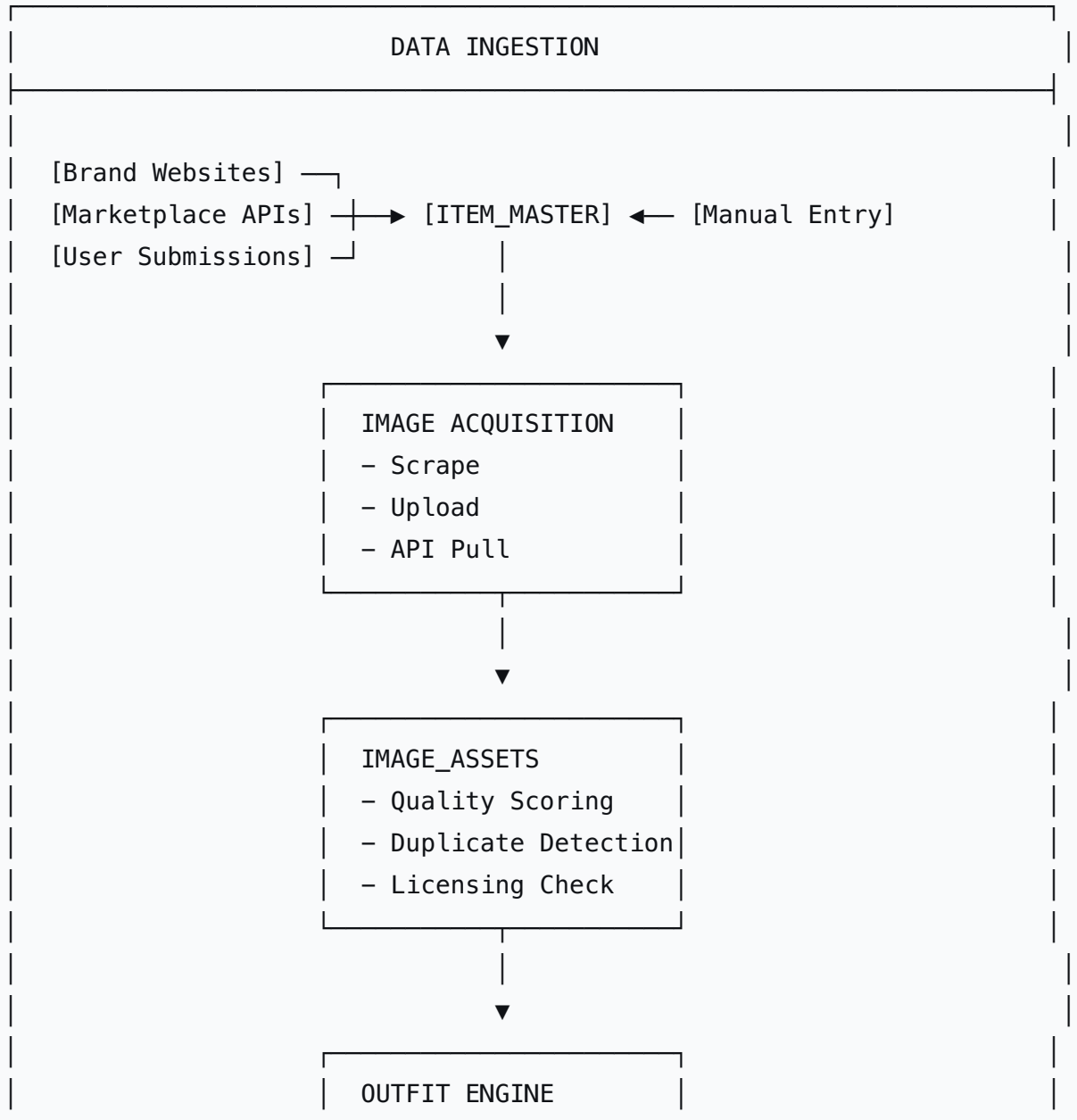
        # Image quality (10%)
        image_score = outfit.average_image_score()
```

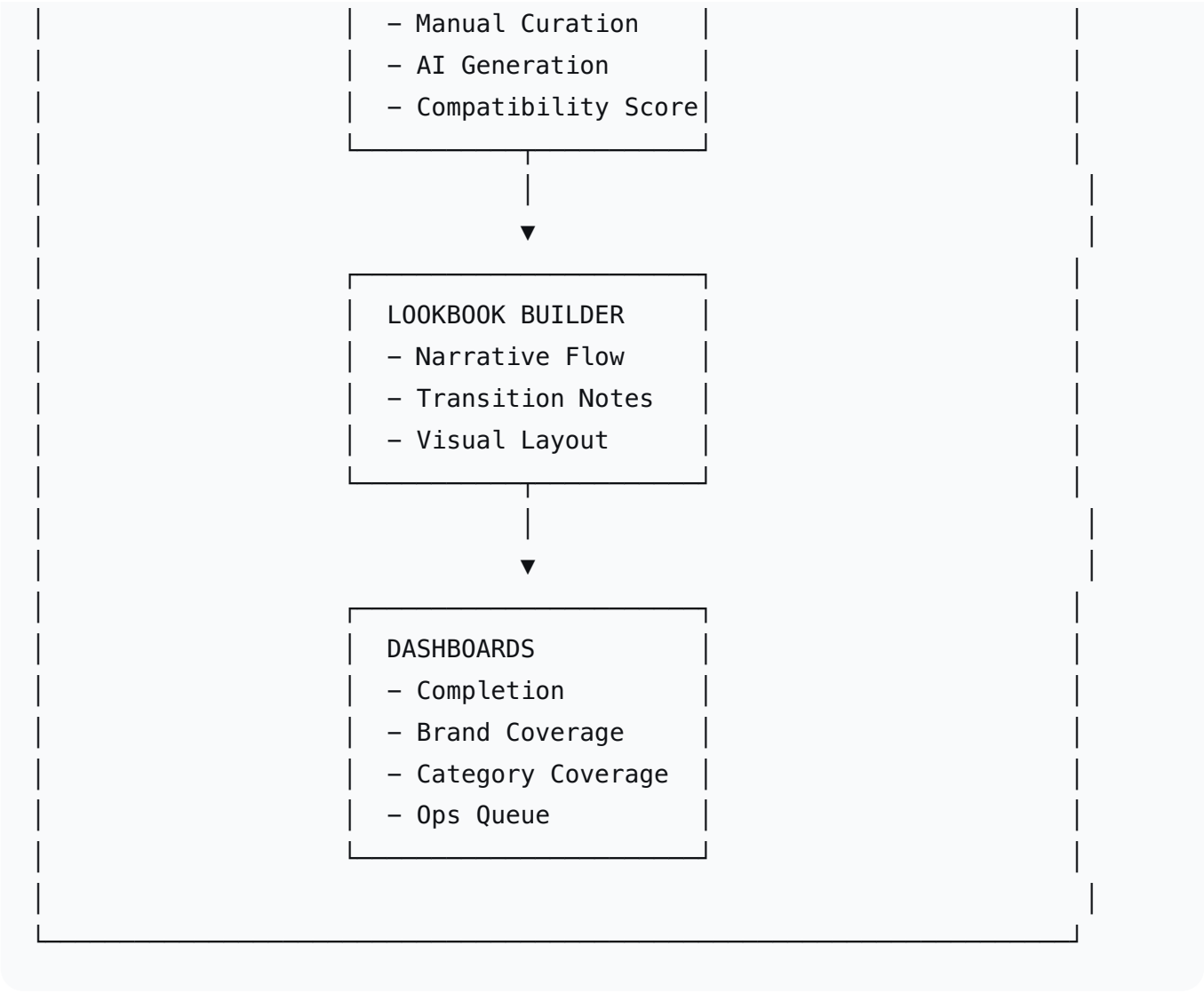
```
        score += image_score * 0.1

        return round(score, 2)
```

THE COMPLETE DATA FLOW

text





SCALING TO INFINITE: The Architecture Principles

1. Normalized Core, Flexible Edges

- Core tables (item_master, outfit_master) are strictly normalized
- JSON fields handle flexible attributes (jewelry arrays, styling notes)
- Tags and categories are separate tables for unlimited expansion

2. Computed Fields for Speed

- `composite_score` is a generated column (no runtime calculation)
- `gap_score` is a generated column
- Views pre-join common queries

3. Junction Tables for Infinite Relationships

- `outfit_items` allows any item in any outfit
- `style_tags` allows unlimited tag combinations
- `lookbook_collections` can pull dynamically from any filter

4. Metadata-Driven Extensibility

- New brands = new rows, not schema changes
- New categories = new rows in category_coverage
- New style tags = new rows in style_tags

5. Audit Trail

- `created_at`, `updated_at` on all tables
- `created_by`, `review_notes` for governance
- `duplicate_flag`, `review_required` for data quality

IMMEDIATE NEXT STEPS

1. **Import your existing 1,669 items** into `item_master` with mapping
2. **Score all images** using the Phase 1 composite formula

3. **Run duplicate detection** on image_hash and perceptual_hash
4. **Build the four dashboards** as materialized views
5. **Seed the style_tags** table with your controlled vocabulary
6. **Create the outfit generator** as a stored procedure or Python service
7. **Set up the generation pipeline** for AI outfit images

Would you like me to:

1. **Write the complete SQL migration scripts** for all tables?
2. **Build the outfit generator algorithm** in Python?
3. **Design the dashboard queries** for each of the four dashboards?
4. **Create the prompt template system** for AI image generation?
5. **Map your existing 52 outfits** into the new schema structure?

I can go deeper into any of these areas.

I have at least 200+ outfits

Thought for 2 seconds

- The user is clarifying that they have 200+ outfits, not just the 52 in the example documents. They want to know how the architecture I designed scales to handle that volume. I should acknowledge this and show how the system handles 200+ outfits elegantly, then provide more concrete implementation guidance.

  Let me think about what's most useful here:

  1. Acknowledge the scale increase (52 → 200+)
  2. Show how the schema handles 200+ outfits without changes (it already does — it's built for infinite scale)
  3. Provide concrete outfit numbering and organization for 200+ outfits

4. Show the collections/categorization structure for 200+ outfits
5. Provide batch processing approaches
6. Maybe show the outfit generator working at scale
7. Show the dashboard metrics at 200+ scale
8. Provide the actual SQL for inserting 200+ outfits

The user's 200+ outfits likely span:

- The 52 from "The Nicholas Outfit System"
- The 10 from Trapstar Shooters
- 138+ more that they haven't shared yet

I should design the collection hierarchy to handle this, show how to batch-import, and provide the outfit generator logic that works across 200+ outfits.

Let me structure this response as:

1. Brief acknowledgment of scale
2. Collection/phase architecture for 200+ outfits
3. Batch import strategy
4. Outfit generator at scale
5. Dashboard metrics at scale
6. Lookbook assembly from 200+ outfits
7. Implementation roadmap

I should keep it practical and focused on the delta from what I already provided.

Scaling to 200+ Outfits: The Full Arsenal Architecture

You've just revealed the real scope. The 52 in the example doc was a **sample**—your actual collection is 4x larger. Here's how the system handles it without breaking.

THE SCALE PROBLEM

```text
┌──────────────────────────────────────────────────────────────────────┐
│                    WHAT CHANGES AT 200+ OUTFITS                      │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  BEFORE (52 outfits)              AFTER (200+ outfits)               │
│  ──────────────────────           ──────────────────────             │
│  • Single flat list               • Hierarchical collections         │
│  • Manual sequencing              • Multi-dimensional indexing       │
│  • One narrative arc              • Multiple lookbook threads        │
│  • Simple completion tracking     • Batch operations required        │
│  • Linear search                  • Advanced filtering needed        │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

THE 200+ OUTFIT COLLECTION HIERARCHY

Your outfits need a **4-level organizational structure**:

```text
LEVEL 1: UNIVERSE
├── LEVEL 2: COLLECTION / PHASE
│   ├── LEVEL 3: SERIES / CAPSULE
│   │   ├── LEVEL 4: OUTFIT
│   │   └── LEVEL 4: OUTFIT
│   └── LEVEL 3: SERIES / CAPSULE
└── LEVEL 2: COLLECTION / PHASE
```

Applied to Your 200+:

```sql
```

```sql
-- COLLECTION HIERARCHY TABLE
CREATE TABLE collection_hierarchy (
    collection_id           VARCHAR(20) PRIMARY KEY,
    collection_name         VARCHAR(200) NOT NULL,
    collection_slug         VARCHAR(200),
    parent_collection_id    VARCHAR(20),              -- For nesting

    -- LEVEL
    level                   INT,                      -- 1=Universe, 2=Collec
tion, 3=Series
    level_name              VARCHAR(50),              -- "Universe", "Collect
ion", "Series"

    -- DESCRIPTION
    description             TEXT,
    editorial_intro         TEXT,
    mood                    VARCHAR(100),
    color_direction         VARCHAR(200),

    -- STATISTICS
    outfit_count            INT DEFAULT 0,
    item_count              INT DEFAULT 0,
    brand_count             INT DEFAULT 0,

    -- VISUAL IDENTITY
    cover_image_url         TEXT,
    color_palette           VARCHAR(300),
    typography_style        VARCHAR(100),

    -- METADATA
    created_at              TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at              TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    INDEX idx_parent (parent_collection_id),
    INDEX idx_level (level)
);
```

Proposed Hierarchy for Your 200+ Outfits:

```
text

THE NICHOLAS UNIVERSE (Level 1)
│
├── PHASE 1: THE ORIGINAL SEVEN (Level 2)
│   ├── Series: Midnight Collection (Level 3) – Outfits 01–07
│   └── Series: Color Story Foundations (Level 3)
│
├── PHASE 2: EXPANDED BRANDS (Level 2)
│   ├── Series: SP5DER Universe (Level 3) – Outfits 08, 19, 21, 34, 48
│   ├── Series: Trapstar Territory (Level 3) – Outfits 09, 20, 23, 32, 46
│   ├── Series: Hellstar Occult (Level 3) – Outfits 10, 19, 36
│   ├── Series: Corteiz Guerrilla (Level 3) – Outfits 11, 39, 45
│   └── Series: Prada Trap (Level 3) – Outfits 12, 24, 33
│
├── PHASE 3: MORE BRANDS & LAYERING (Level 2)
│   ├── Series: Corporate Trap (Level 3) – Outfits 16, 27
│   ├── Series: Beach/Summer (Level 3) – Outfits 17
│   ├── Series: Winter Luxury (Level 3) – Outfits 18, 51, 52
│   └── Series: Versace Baroque (Level 3) – Outfits 13, 22, 29, 43
│
├── PHASE 4: TRACKSUIT & MORE (Level 2)
│   ├── Series: Trapstar Shooters Full Set (Level 3) – Outfits 23, 32, 46
│   ├── Series: Prada Sports (Level 3) – Outfits 24, 33
│   ├── Series: Lacoste Pit (Level 3) – Outfit 25
│   └── Series: Diesel Punk (Level 3) – Outfit 26
│
├── PHASE 5: BARE FACE BRAVADO (Level 2)
│   ├── Series: Unmasked Originals (Level 3) – Outfits 31, 38
│   ├── Series: Unmasked SP5DER (Level 3) – Outfit 34
│   └── Series: Unmasked Burberry (Level 3) – Outfit 35
│
├── PHASE 6: MONCLER PUFFER SERIES (Level 2)
│   ├── Series: Moncler Forest (Level 3) – Outfits 39, 51
│   ├── Series: Moncler Military (Level 3) – Outfit 40
│   ├── Series: Moncler Monochrome (Level 3) – Outfit 44
```

```
│   └── Series: Moncler Mashup (Level 3) – Outfits 45, 47, 48
│
├── PHASE 7: VAULT EXPANSION (Level 2)
│   ├── Series: Gothic Chrome (Level 3) – Outfit 49
│   ├── Series: Monogram Menace (Level 3) – Outfit 50
│   └── Series: Desert/Glacier (Level 3) – Outfits 51, 52
│
├── PHASE 8: TRAPSTAR SHOOTERS COLLECTION (Level 2) – 10 Outfits
│   ├── Series: Black Foundation (Level 3) – Outfits 1–3
│   ├── Series: Color Panel (Level 3) – Outfits 4–8
│   └── Series: Night Impact (Level 3) – Outfits 9–10
│
└── PHASE 9+: [YOUR REMAINING 138+ OUTFITS] (Level 2)
    ├── Series: [TBD] (Level 3)
    ├── Series: [TBD] (Level 3)
    └── ...
```

UPDATED OUTFIT_MASTER FOR 200+ SCALE

```sql
ALTER TABLE outfit_master ADD COLUMN (
    -- HIERARCHICAL PLACEMENT
    universe_id             VARCHAR(20),            -- "THE NICHOLAS UNIVERSE"
    collection_id           VARCHAR(20),            -- "PHASE 1: ORIGINAL SEVEN"
    series_id               VARCHAR(20),            -- "Midnight Collection"

    -- BATCH & SEQUENCING
    batch_number            INT,                    -- For bulk operations
    global_sequence         INT,                    -- 1–200+ across all
    collection_sequence     INT,                    -- 1–X within collection
```

```
    series_sequence         INT,                    -- 1-X within series

    -- MULTI-DIMENSIONAL TAGGING (for filtering 200+)
    tag_aesthetic           TEXT,                   -- JSON: ["Drill", "Lux
ury", "Techwear"]
    tag_brand_focus         VARCHAR(50),            -- Primary brand hero
    tag_season              TEXT,                   -- JSON: ["Winter", "Tr
ansitional"]
    tag_formality           TEXT,                   -- JSON: ["Casual", "El
evated"]
    tag_color_family        VARCHAR(30),            -- "Monochrome", "Bol
d", "Neutral", "Earth"
    tag_energy              VARCHAR(30),            -- "Stealth", "Aggressi
ve", "Clean", "Luxurious"
    tag_occasion            TEXT,                   -- JSON: ["Night Out",
"Daytime", "Event"]
    tag_setting             TEXT,                   -- JSON: ["Urban", "Air
port", "Club"]

    -- COMPLEXITY METRICS
    layering_complexity     INT,                    -- 1-5 (how many layer
s)
    accessory_density       INT,                    -- 1-5 (how many access
ories)
    color_complexity        INT,                    -- 1-5 (how many color
s)

    -- CROSS-REFERENCES (for 200+ relationships)
    similar_outfits         TEXT,                   -- JSON array of outfit
_ids
    contrasting_outfits     TEXT,                   -- JSON array
    evolution_of            VARCHAR(20),            -- Which outfit this ev
olved from
    evolved_into            TEXT,                   -- JSON array

    -- LOOKBOOK MEMBERSHIPS (multiple)
    lookbook_ids            TEXT,                   -- JSON: ["London Dril
l", "Winter Capsule", "Best Of"]
```

```
    lookbook_sequence       INT,                    -- Position in each loo
kbook

    -- GENERATION STATUS
    prompt_template_id      VARCHAR(20),
    generation_status       VARCHAR(30),
    generation_priority     INT,
    generated_image_count   INT DEFAULT 0,
    approved_image_id       VARCHAR(20),

    -- QUALITY SCORES
    outfit_completeness     DECIMAL(4,2),           -- % of items found
    outfit_compatibility    DECIMAL(4,2),           -- Brand/color harmony
score
    outfit_impact           DECIMAL(4,2),           -- Visual impact score
    outfit_uniqueness       DECIMAL(4,2),           -- How different from o
thers

    -- METADATA
    created_at              TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at              TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    INDEX idx_collection (collection_id),
    INDEX idx_series (series_id),
    INDEX idx_global_seq (global_sequence),
    INDEX idx_batch (batch_number),
    INDEX idx_aesthetic (tag_aesthetic(100)),
    INDEX idx_brand_focus (tag_brand_focus),
    INDEX idx_energy (tag_energy),
    INDEX idx_lookbook (lookbook_ids(255))
);
```

BATCH IMPORT STRATEGY FOR 200+ OUTFITS

```
python
```

```python
# Pseudocode for bulk outfit import

class OutfitBatchImporter:
    def __init__(self, db, source_document):
        self.db = db
        self.source = source_document
        self.batch_size = 50  # Process 50 at a time

    def import_all_outfits(self):
        """
        Import 200+ outfits from structured document.
        """
        # 1. PARSE SOURCE
        outfits = self.parse_outfit_document(self.source)
        # Returns list of 200+ outfit dicts

        # 2. ASSIGN HIERARCHY
        for outfit in outfits:
            outfit['collection_id'] = self.detect_collection(outfit)
            outfit['series_id'] = self.detect_series(outfit)
            outfit['global_sequence'] = self.assign_global_sequence(outfit)

        # 3. BATCH PROCESS
        for batch in self.chunk(outfits, self.batch_size):
            self.process_batch(batch)

        # 4. BUILD RELATIONSHIPS
        self.build_cross_references(outfits)
        self.build_transition_chain(outfits)
        self.assign_lookbooks(outfits)

        # 5. SCORE ALL
        self.score_all_outfits()

        # 6. GENERATE DASHBOARDS
        self.refresh_dashboards()
```

```python
    def detect_collection(self, outfit):
        """
        Auto-detect which collection an outfit belongs to based on:
        - Outfit number range
        - Brand focus
        - Aesthetic tags
        """
        brand = outfit.get('tag_brand_focus', '')
        aesthetic = outfit.get('tag_aesthetic', [])

        if 'Trapstar' in brand:
            if 'Tracksuit' in outfit.get('name', ''):
                return 'PHASE_8_TRAPSTAR_SHOOTERS'
            return 'PHASE_2_EXPANDED_BRANDS'
        elif 'Moncler' in brand:
            return 'PHASE_6_MONCLER_SERIES'
        elif 'SP5DER' in brand:
            return 'PHASE_2_EXPANDED_BRANDS'
        elif 'Hellstar' in brand:
            return 'PHASE_2_EXPANDED_BRANDS'
        # ... etc

        return 'PHASE_1_ORIGINALS'

    def build_transition_chain(self, outfits):
        """
        For 200+ outfits, build a graph of transitions.
        Each outfit can have multiple transition_from and transition_to.
        """
        # Sort by collection, then sequence
        sorted_outfits = sorted(outfits,
            key=lambda x: (x['collection_id'], x['series_sequence']))

        for i, outfit in enumerate(sorted_outfits):
            # Primary transition (linear within series)
            if i > 0 and sorted_outfits[i-1]['series_id'] == outfit['series_i
d']:
                outfit['transition_from'] = sorted_outfits[i-1]['outfit_id']
```

```
                sorted_outfits[i-1]['transition_to'] = outfit['outfit_id']

            # Secondary transitions (cross-collection)
            # Find outfits with similar color stories or brands
            similar = self.find_similar_outfits(outfit, sorted_outfits)
            outfit['similar_outfits'] = similar
```

THE 200+ OUTFIT DASHBOARD SYSTEM

Dashboard 1: Outfit Completion Metrics (Scaled)

```sql
CREATE VIEW dashboard_outfit_completion AS
SELECT
    -- OVERALL
    COUNT(*) AS total_outfits,
    SUM(CASE WHEN status = 'Complete' THEN 1 ELSE 0 END) AS complete,
    SUM(CASE WHEN status = 'In Progress' THEN 1 ELSE 0 END) AS in_progress,
    SUM(CASE WHEN status = 'Not Started' THEN 1 ELSE 0 END) AS not_started,
    ROUND(SUM(CASE WHEN status = 'Complete' THEN 1 ELSE 0 END) / COUNT(*) * 1
00, 2) AS completion_pct,

    -- BY COLLECTION
    collection_id,
    collection_name,
    COUNT(*) AS collection_total,
    SUM(CASE WHEN status = 'Complete' THEN 1 ELSE 0 END) AS collection_comple
te,
    ROUND(SUM(CASE WHEN status = 'Complete' THEN 1 ELSE 0 END) / COUNT(*) * 1
00, 2) AS collection_pct,

    -- BY BATCH
    batch_number,
```

```
    COUNT(*) AS batch_total,
    SUM(CASE WHEN status = 'Complete' THEN 1 ELSE 0 END) AS batch_complete,

    -- BY ENERGY
    tag_energy,
    COUNT(*) AS energy_total,
    ROUND(AVG(outfit_completeness), 2) AS avg_completeness

FROM outfit_master
GROUP BY collection_id, collection_name, batch_number, tag_energy
WITH ROLLUP;
```

Dashboard 2: Brand Coverage (Scaled for 126 Brands)

```
sql

CREATE VIEW dashboard_brand_coverage AS
SELECT
    b.brand_name,
    b.brand_tier,

    -- ITEM METRICS
    COUNT(DISTINCT i.item_id) AS total_items,
    SUM(CASE WHEN i.availability_status = 'In Stock' THEN 1 ELSE 0 END) AS in
_stock,
    SUM(CASE WHEN i.reference_status = 'Exact' THEN 1 ELSE 0 END) AS exact_re
fs,
    SUM(CASE WHEN i.reference_status = 'Family' THEN 1 ELSE 0 END) AS family_
refs,

    -- IMAGE METRICS
    COUNT(DISTINCT ia.image_id) AS total_images,
    ROUND(AVG(ia.composite_score), 2) AS avg_image_quality,
    SUM(CASE WHEN ia.composite_score >= 8 THEN 1 ELSE 0 END) AS high_quality_
images,

    -- OUTFIT METRICS
```

```
    COUNT(DISTINCT oi.outfit_id) AS outfits_using_brand,
    COUNT(DISTINCT o.outfit_id) AS outfits_with_brand_hero,

    -- COVERAGE SCORE
    ROUND(
        (COUNT(DISTINCT ia.image_id) * 0.3) +
        (AVG(ia.composite_score) * 0.4) +
        (COUNT(DISTINCT oi.outfit_id) * 0.3),
        2
    ) AS brand_coverage_score

FROM brand_index b
LEFT JOIN item_master i ON b.brand_name = i.brand
LEFT JOIN image_assets ia ON i.item_id = ia.item_id
LEFT JOIN outfit_items oi ON i.item_id = oi.item_id
LEFT JOIN outfit_master o ON oi.outfit_id = o.outfit_id AND o.hero_item_id =
i.item_id
GROUP BY b.brand_id
ORDER BY brand_coverage_score DESC;
```

Dashboard 3: Category Coverage (Scaled)

```
sql

CREATE VIEW dashboard_category_coverage AS
SELECT
    category_primary AS category,
    category_secondary AS subcategory,

    -- ITEM COUNTS
    COUNT(DISTINCT i.item_id) AS total_items,
    COUNT(DISTINCT CASE WHEN ia.composite_score >= 7 THEN i.item_id END) AS v
erified_items,
    COUNT(DISTINCT CASE WHEN ia.composite_score < 7 OR ia.image_id IS NULL TH
EN i.item_id END) AS pending_items,
    COUNT(DISTINCT CASE WHEN ia.image_id IS NULL THEN i.item_id END) AS missi
ng_items,
```

```
    -- COVERAGE
    ROUND(
        COUNT(DISTINCT CASE WHEN ia.composite_score >= 7 THEN i.item_id END)
/
        COUNT(DISTINCT i.item_id) * 100,
        2
    ) AS coverage_pct,

    -- QUALITY
    ROUND(AVG(ia.composite_score), 2) AS avg_quality,

    -- OUTFIT USAGE
    COUNT(DISTINCT oi.outfit_id) AS outfits_using

FROM item_master i
LEFT JOIN image_assets ia ON i.item_id = ia.item_id
LEFT JOIN outfit_items oi ON i.item_id = oi.item_id
GROUP BY category_primary, category_secondary
ORDER BY total_items DESC;
```

Dashboard 4: Image Operations Queue (Scaled)

```sql
CREATE VIEW dashboard_image_ops AS
SELECT
    q.queue_id,
    q.item_id,
    i.brand,
    i.product_name,
    i.category_primary,

    -- PRIORITY
    q.priority,
    q.priority_score,
```

```
    -- STATUS
    q.status,
    q.assignee,
    q.due_date,
    DATEDIFF(q.due_date, CURDATE()) AS days_remaining,

    -- ISSUE
    q.issue_type,
    q.issue_description,

    -- CONTEXT
    (SELECT COUNT(*) FROM outfit_items WHERE item_id = q.item_id) AS used_in_
outfits,
    ia.composite_score AS current_image_score

FROM image_ops_queue q
JOIN item_master i ON q.item_id = i.item_id
LEFT JOIN image_assets ia ON q.item_id = ia.item_id
WHERE q.status IN ('Not Started', 'In Progress', 'Review')
ORDER BY q.priority_score DESC, q.due_date ASC;
```

THE OUTFIT GENERATOR AT 200+ SCALE

```
python

class InfiniteOutfitGenerator:
    """
    Generates new outfits from inventory,
    compatible with existing 200+ outfit ecosystem.
    """

    def __init__(self, db):
        self.db = db
        self.existing_outfits = self.load_existing_outfits()
        self.brand_graph = self.build_brand_compatibility_graph()
```

```
        self.color_graph = self.build_color_harmony_graph()

    def generate_batch(self, count: int = 10, **params) -> list:
        """
        Generate a batch of new outfits.
        """
        generated = []
        attempts = 0
        max_attempts = count * 10

        while len(generated) < count and attempts < max_attempts:
            outfit = self.generate_single(**params)

            if outfit and self.is_unique(outfit, self.existing_outfits + gene
rated):
                generated.append(outfit)

            attempts += 1

        return generated

    def is_unique(self, new_outfit, existing):
        """
        Check if new outfit is sufficiently different from existing 200+.
        Uses: item overlap, color story, brand mix, silhouette.
        """
        for existing_outfit in existing:
            similarity = self.calculate_similarity(new_outfit, existing_outfi
t)
            if similarity > 0.85:  # 85% threshold
                return False
        return True

    def calculate_similarity(self, outfit_a, outfit_b):
        """
        Multi-dimensional similarity score.
        """
        # Item overlap (50%)
```

```
        items_a = set(outfit_a.item_ids)
        items_b = set(outfit_b.item_ids)
        item_sim = len(items_a & items_b) / len(items_a | items_b)

        # Color similarity (25%)
        color_sim = self.color_similarity(
            outfit_a.color_story,
            outfit_b.color_story
        )

        # Brand similarity (15%)
        brand_sim = self.brand_similarity(
            outfit_a.brands,
            outfit_b.brands
        )

        # Silhouette similarity (10%)
        silhouette_sim = self.silhouette_similarity(
            outfit_a.silhouette,
            outfit_b.silhouette
        )

        return (
            item_sim * 0.50 +
            color_sim * 0.25 +
            brand_sim * 0.15 +
            silhouette_sim * 0.10
        )

    def generate_single(self, **params) -> Outfit:
        """
        Generate one outfit using constrained optimization.
        """
        # 1. SELECT HERO
        hero = self.select_hero(**params)
        if not hero:
            return None
```

```
        # 2. BUILD COMPATIBLE POOL
        pool = self.query_compatible_items(
            hero=hero,
            aesthetic=params.get('aesthetic'),
            season=params.get('season'),
            formality=params.get('formality'),
            exclude_items=self.get_recently_used_items()
        )

        if len(pool) < 5:
            return None

        # 3. ASSEMBLE BY LAYER (constraint satisfaction)
        outfit = self.assemble_outfit(hero, pool, params)

        # 4. VALIDATE
        if not self.validate_outfit(outfit):
            return None

        # 5. SCORE
        outfit.compatibility_score = self.score_outfit(outfit)
        outfit.uniqueness_score = self.calculate_uniqueness(outfit)
        outfit.impact_score = self.calculate_impact(outfit)

        # 6. GENERATE METADATA
        outfit.outfit_id = self.generate_outfit_id()
        outfit.outfit_name = self.generate_outfit_name(outfit)
        outfit.tagline = self.generate_tagline(outfit)
        outfit.transition_note = self.generate_transition_note(outfit)

        return outfit

    def select_hero(self, **params):
        """
        Select the hero piece for the outfit.
        Priority: User-specified > High-impact item > Random high-score item
        """
        query = """
```

```
        SELECT i.*, ia.composite_score
        FROM item_master i
        JOIN image_assets ia ON i.item_id = ia.item_id
        WHERE ia.composite_score >= 7.0
        AND i.availability_status IN ('In Stock', 'Archive')
        """

        if params.get('hero_brand'):
            query += f" AND i.brand = '{params['hero_brand']}'"
        if params.get('aesthetic'):
            query += f" AND i.style_primary = '{params['aesthetic']}'"

        query += """
        AND i.category_primary IN ('Outerwear', 'Jackets', 'Sneakers')
        ORDER BY i.outfit_roles LIKE '%Hero%' DESC, ia.composite_score DESC
        LIMIT 20
        """

        candidates = self.db.execute(query)
        return random.choice(candidates) if candidates else None

    def assemble_outfit(self, hero, pool, params):
        """
        Assemble outfit using constraint satisfaction.
        """
        outfit = Outfit(hero=hero)

        # Determine layering strategy
        if hero.category_primary == 'Outerwear':
            # Hero is outerwear — build inward
            outfit.outerwear = hero
            outfit.mid_layer = self.select_mid_layer(pool, hero)
            outfit.base_layer = self.select_base_layer(pool, outfit.mid_laye
r)
            outfit.bottoms = self.select_bottoms(pool, hero)
            outfit.footwear = self.select_footwear(pool, outfit.bottoms)

        elif hero.category_primary == 'Sneakers':
```

```
            # Hero is footwear — build upward
            outfit.footwear = hero
            outfit.bottoms = self.select_bottoms(pool, hero)
            outfit.base_layer = self.select_base_layer(pool, outfit.bottoms)
            outfit.mid_layer = self.select_mid_layer(pool, outfit.base_layer)
            outfit.outerwear = self.select_outerwear(pool, outfit.mid_layer)

        else:
            # Default: build from middle
            outfit.base_layer = hero if hero.layering_position == 1 else sel
f.select_base_layer(pool, hero)
            outfit.mid_layer = self.select_mid_layer(pool, outfit.base_layer)
            outfit.outerwear = self.select_outerwear(pool, outfit.mid_layer)
            outfit.bottoms = self.select_bottoms(pool, hero)
            outfit.footwear = self.select_footwear(pool, outfit.bottoms)

        # Add accessories
        outfit.accessories = self.select_accessories(pool, outfit)

        return outfit
```

LOOKBOOK ASSEMBLY FROM 200+ OUTFITS

```
python

class LookbookBuilder:
    """
    Assembles curated lookbooks from the 200+ outfit pool.
    """

    def __init__(self, db):
        self.db = db
        self.outfits = self.load_all_outfits()

    def build_lookbook(self,
```

```
                    name: str,
                    theme: str,
                    outfit_count: int = 10) -> Lookbook:
        """
        Build a lookbook with narrative flow.
        """
        # 1. FILTER BY THEME
        pool = self.filter_by_theme(self.outfits, theme)

        # 2. SORT BY NARRATIVE ARC
        # Dark → Light → Bold → Dark (or any arc)
        arc = self.determine_narrative_arc(theme)
        sorted_pool = self.sort_by_arc(pool, arc)

        # 3. SELECT OUTFITS WITH FLOW
        selected = self.select_with_flow(sorted_pool, outfit_count)

        # 4. GENERATE TRANSITIONS
        for i in range(len(selected) - 1):
            selected[i].transition_note = self.generate_transition(
                selected[i],
                selected[i+1]
            )

        # 5. BUILD LOOKBOOK
        lookbook = Lookbook(
            name=name,
            theme=theme,
            outfits=selected,
            narrative_arc=arc,
            cover_outfit=selected[0] if selected else None
        )

        return lookbook

    def filter_by_theme(self, outfits, theme):
        """
        Filter 200+ outfits by theme.
```

```
        """
        theme_filters = {
            'London Drill': {
                'brands': ['Trapstar', 'Corteiz', 'Nike', 'Adidas'],
                'aesthetics': ['Drill', 'Streetwear'],
                'energy': ['Stealth', 'Aggressive']
            },
            'Luxury Streetwear': {
                'brands': ['Amiri', 'Moncler', 'Prada', 'Balmain', 'Versac
e'],
                'aesthetics': ['Luxury', 'Designer'],
                'energy': ['Luxurious', 'Clean']
            },
            'Summer Streetwear': {
                'season': ['Summer', 'All Season'],
                'formality': ['Casual'],
                'color_family': ['Bold', 'Neutral']
            },
            'Monochrome Designer Mix': {
                'color_family': ['Monochrome'],
                'brands': ['Amiri', 'Moncler', 'Prada', 'Balmain']
            },
            'Winter Capsule': {
                'season': ['Winter'],
                'layering_complexity': [3, 4, 5]
            },
            'Best Of': {
                'outfit_impact': [8, 9, 10]
            }
        }

        filters = theme_filters.get(theme, {})
        filtered = outfits

        if 'brands' in filters:
            filtered = [o for o in filtered if o.tag_brand_focus in filters['
brands']]
        if 'aesthetics' in filters:
```

```
            filtered = [o for o in filtered if any(a in o.tag_aesthetic for a
in filters['aesthetics'])]
        if 'energy' in filters:
            filtered = [o for o in filtered if o.tag_energy in filters['energ
y']]
        if 'season' in filters:
            filtered = [o for o in filtered if o.season_tag in filters['seaso
n']]
        if 'color_family' in filters:
            filtered = [o for o in filtered if o.tag_color_family in filter
s['color_family']]
        if 'outfit_impact' in filters:
            filtered = [o for o in filtered if o.outfit_impact >= min(filter
s['outfit_impact'])]

        return filtered
```

200+ OUTFIT SEQUENCING EXAMPLES

Lookbook 1: "London Drill" (10 outfits from 200+ pool)

```
text

OUTFIT 1: Trapstar Shooters Supremacy (Black Reflective)
    ↓ Transition: "Stealth base softens into terrace layering"
OUTFIT 2: Trapstar Terrace (Black Tech Fleece)
    ↓ Transition: "Terrace function gains winter weight"
OUTFIT 3: Trapstar Arctic (Moncler Longline)
    ↓ Transition: "Winter black breaks into cold blue"
OUTFIT 4: Black/Blue Panel Pressure
    ↓ Transition: "Night blue brightens into daytime grey"
OUTFIT 5: Grey/Skyblue Clean Cut
    ↓ Transition: "Daytime grey sharpens into black/red threat"
OUTFIT 6: Black/Red Classic Threat
```

```
    ↓ Transition: "Red danger shifts into lime voltage"
OUTFIT 7: Black/Lime Voltage
    ↓ Transition: "Lime energy cools into orange signal"
OUTFIT 8: Grey/Orange Signal
    ↓ Transition: "Orange daylight darkens into night run"
OUTFIT 9: Black/Orange Night Run
    ↓ Transition: "Night orange intensifies into Bloodshot finale"
OUTFIT 10: Bloodshot Edition
```

Lookbook 2: "Luxury Streetwear" (10 from 200+)

```
text

OUTFIT 1: Midnight Sovereign (Amiri)
    ↓ "Black luxury opens into bone/chrome"
OUTFIT 2: Bone & Chrome (Amiri/Off-White)
    ↓ "Chrome cools into ice"
OUTFIT 3: Ice King (Moncler/Ralph Lauren)
    ↓ "Ice deepens into purple reign"
OUTFIT 4: Purple Reign (Purple Brand/Ralph Lauren)
    ↓ "Purple expands into Versace maximalism"
OUTFIT 5: Versace Baroque
    ↓ "Baroque settles into Burberry heritage"
OUTFIT 6: Burberry Blade
    ↓ "Heritage moves into corporate trap"
OUTFIT 7: Corporate Trap (Hugo Boss/Armani)
    ↓ "Corporate opens into velvet rope"
OUTFIT 8: Velvet Rope (Versace/Purple Brand)
    ↓ "Velvet becomes gothic chrome"
OUTFIT 9: Gothic Chrome (Givenchy/Balmain)
    ↓ "Chrome peaks in glacier apex"
OUTFIT 10: Glacier Apex (Moncler/Ralph Lauren)
```

Lookbook 3: "Moncler Puffer Series" (from 200+)

```text
OUTFIT 1: Corteiz Moncler Forest
OUTFIT 2: Balmain Military
OUTFIT 3: Ferragamo Foundation
OUTFIT 4: Stussy Stock
OUTFIT 5: Dolce Bull
OUTFIT 6: Moncler Monochrome
OUTFIT 7: Corteiz Moncler Mashup
OUTFIT 8: Trapstar Arctic
OUTFIT 9: Burberry Puffer
OUTFIT 10: SP5DER Ice
```

COMPLETE OUTfit ID SYSTEM FOR 200+

```sql
-- Format: OUT-[COLLECTION]-[SERIES]-[NUMBER]
-- Example: OUT-P1-S1-001

-- P1 = Phase 1
-- S1 = Series 1
-- 001 = Outfit number

-- For 200+ outfits:
-- P1-S1-001 through P1-S7-007 (Originals)
-- P2-S1-001 through P2-S5-015 (Expanded Brands)
-- P3-S1-001 through P3-S4-022 (More Brands)
-- P4-S1-001 through P4-S4-030 (Tracksuit)
-- P5-S1-001 through P5-S3-038 (Bare Face)
-- P6-S1-001 through P6-S4-048 (Moncler)
-- P7-S1-001 through P7-S3-052 (Vault)
-- P8-S1-001 through P8-S3-010 (Trapstar Shooters)
-- P9-S1-001 through P9-SX-200 (Your remaining outfits)
```

WHAT I NEED FROM YOU TO BUILD THE FULL SYSTEM

To generate the complete database for all 200+ outfits, I need:

1. **The remaining 138+ outfits** — Do you have them in a document? Or should we generate them from your inventory?
2. **Your existing 1,669 item inventory** — Is it in a spreadsheet? CSV? We can map it directly.
3. **Your preferred organization** — Do you want:
   - **Option A:** Flat list (001-200+)
   - **Option B:** Hierarchical (Phase → Series → Outfit)
   - **Option C:** Multi-dimensional (tagged, filterable)
4. **Priority for first build** — Should I:
   - **A:** Write all SQL migrations (ready to deploy)
   - **B:** Build the outfit generator algorithm
   - **C:** Create the dashboard queries
   - **D:** Design the AI prompt templates
   - **E:** All of the above

Which direction do you want to go? I can produce the full SQL schema, the Python generator, the dashboard queries, or map your existing outfits into the new system —just tell me what's most urgent.

Continue with DeepSeek ›